

Unidad 2:

UADE



Operaciones de Ingeniería de Software (Parte I)

Dra. Ing. Roxana Martínez
(Docente Investigadora)

Unidad 2 - Operaciones de Ingeniería de Software

Temario:

- *Procesos de operación.*
- *Instalación de paquetes de software.*
- *Scripting y automatización de instalación.*
- *Pruebas y solución de problemas.*
- *Performance, Reliability y Balanceo de Carga.*
- *Plan de operaciones. Entornos de desarrollo, testing y producción.*
- *Disponibilidad de software, continuidad y niveles de servicio.*
Dimensionamiento y gestión de capacidad.



Procesos de operación



Ing. de Software

Procesos de operación en IS

Las operaciones en ingeniería de software se refieren a las actividades relacionadas con la ***gestión, el mantenimiento y el soporte continuo de sistemas de software una vez que han sido desarrollados e implementados.***



Procesos de operación en IS



Estas actividades garantizan que el software funcione correctamente, cumpla con los requisitos del usuario y se adapte a los cambios en el entorno operativo.

Actividades - Operación en IS

Despliegue del software: Este proceso implica la instalación del software en el entorno operativo previsto.

Puede incluir la configuración de servidores, bases de datos y otros componentes necesarios para el funcionamiento del software.



Actividades - Operación en IS

Monitoreo del sistema: Es fundamental monitorear el rendimiento del sistema de software en producción para identificar problemas potenciales y garantizar un funcionamiento óptimo.

Esto implica el seguimiento de métricas como el uso de recursos, el tiempo de respuesta y la disponibilidad del sistema.



Ejercicios en Grupo

EJERCICIO EN EQUIPO Nro. 01: CASO DE APLICACIÓN: Métricas Monitoreo de Software



¿QUÉ MÉTRICAS SE PODRÍAN IMPLEMENTAR PARA
EL MONITOREO DEL SOFTWARE?

Actividades - Operación en IS

Gestión de incidencias: proceso de gestionar y resolver incidentes que surgen en la operación del software.

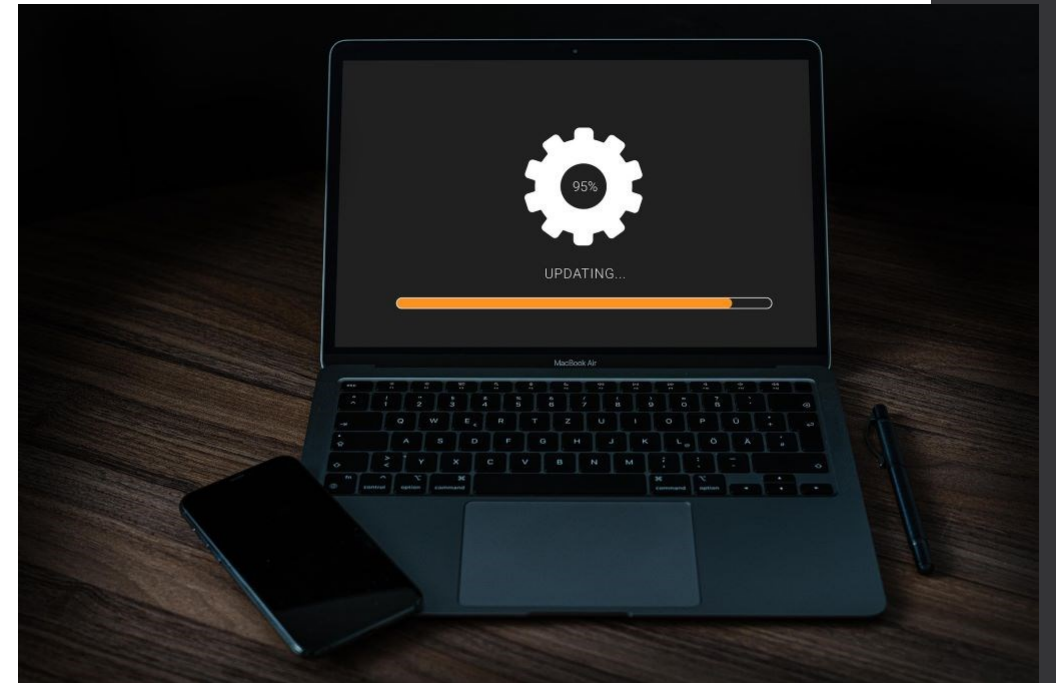
Puede incluir la identificación, el registro, la priorización, la asignación y la resolución de problemas que afectan el rendimiento o la disponibilidad del sistema.



Actividades - Operación en IS

Actualizaciones y parches: Los sistemas de software requieren actualizaciones periódicas para corregir errores, agregar nuevas características y mejorar el rendimiento y la seguridad.

Este proceso implica la planificación, prueba e implementación de actualizaciones y parches de software en el entorno operativo.



Actividades - Operación en IS

Gestión de configuraciones:

Consiste en administrar las configuraciones del sistema de software para garantizar la coherencia y la integridad en todos los entornos. Esto implica el control de versiones, la gestión de cambios y la documentación de las configuraciones del sistema.



Actividades - Operación en IS

Respaldos y recuperación: Es esencial realizar copias de seguridad periódicas de los datos y configuraciones del sistema para garantizar la disponibilidad y la integridad de la información en caso de fallos o desastres.

Este proceso también incluye la planificación y la ejecución de procedimientos de recuperación en caso de pérdida de datos o fallas del sistema.



Actividades - Operación en IS



Optimización del rendimiento:

Implica la identificación y la implementación de mejoras en el rendimiento del sistema de software para garantizar una experiencia de usuario óptima.

Esto puede incluir la optimización de consultas de base de datos, la mejora de algoritmos y la optimización de la infraestructura de hardware.

DEBATE:

¿Qué implica la integridad en todos los entornos de software?



Instalación de paquetes de software



Ing. de Software

Instalación de paquetes de Software

Requisitos previos a la instalación de software

Antes de realizar la instalación, asegúrese de que haya preparado el sistema y recopilado toda la información necesaria.

Consultar las Notas de la versión y comprobar los requisitos del sistema

Consulte las Notas de la versión, y compruebe que el sistema y el hardware sean compatibles con el producto que desee instalar.

- Requisitos del sistema
- Hardware informático certificado y recomendado

Instalación de paquetes de Software



Aspecto	Windows	macOS	Linux (Ubuntu/Debian)
Compatibilidad del sistema operativo	Verificar la compatibilidad con la versión de Windows.	Asegurarse de que el software sea compatible con macOS.	Comprobar la compatibilidad con la distribución y versión de Linux.
Requisitos del sistema	Consultar los requisitos de hardware y software.	Revisar los requisitos mínimos de sistema.	Verificar los requisitos mínimos de sistema.
Origen del software	Descargar desde el sitio web oficial o tiendas de aplicaciones.	Preferiblemente obtenerlo del sitio web oficial del desarrollador.	Descargar desde repositorios oficiales o sitios confiables.
Integridad del paquete	Verificar la integridad del archivo descargado.	Comprobar la autenticidad mediante sumas de verificación.	Usar sumas de verificación o firmas digitales.

Aspecto	Windows	macOS	Linux (Ubuntu/Debian)
Privilegios de administrador	Posiblemente se requieran privilegios de administrador.	Se pueden necesitar permisos de administrador para la instalación.	Generalmente se requieren privilegios de sudo o root.
Configuración durante la instalación	Revisar y personalizar opciones de instalación según preferencias.	Leer y ajustar las configuraciones durante la instalación.	Personalizar las opciones de instalación según preferencias.
Actualizaciones y parches	Mantener actualizado el software mediante actualizaciones automáticas.	Instalar actualizaciones de software periódicamente.	Actualizar el sistema y el software regularmente.
Documentación y soporte	Consultar la documentación oficial y buscar soporte en línea.	Acceder a la documentación del software y buscar ayuda en línea.	Utilizar la documentación oficial y la comunidad de usuarios para obtener soporte.



Ejercicios en Grupo

EJERCICIO EN EQUIPO Nro. 02:

Investiguen que tipos de software de gestión de incidencias existen en el mercado actual. Luego, seleccionen uno como equipo y describan sus principales ventajas y funcionalidades en un ppt/pdf para exponerlo en 5 min aprox. por equipo.



Scripting y automatización de instalación



Scripting

El scripting y la automatización de la instalación de software son prácticas que pueden ahorrar tiempo y minimizar errores al implementar software en múltiples sistemas.



Herramientas: Scripting

El scripting implica escribir secuencias de comandos o programas pequeños que realizan tareas específicas de manera automatizada.

Algunos lenguajes de scripting populares para este propósito incluyen:

Bash: Ideal para sistemas basados en Unix/Linux, como macOS y distribuciones de Linux.

PowerShell: Utilizado principalmente en entornos Windows.

Python: Versátil y multiplataforma, puede utilizarse en una variedad de sistemas operativos.



Herramientas:

Herramientas de gestión de configuración:

Permiten definir y mantener el estado deseado de la configuración de un sistema de forma automática.

Algunas herramientas incluyen:

Ansible: Permite automatizar la implementación, la administración y la configuración de sistemas.

Chef: Facilita la configuración, el aprovisionamiento y la gestión de infraestructuras a gran escala.

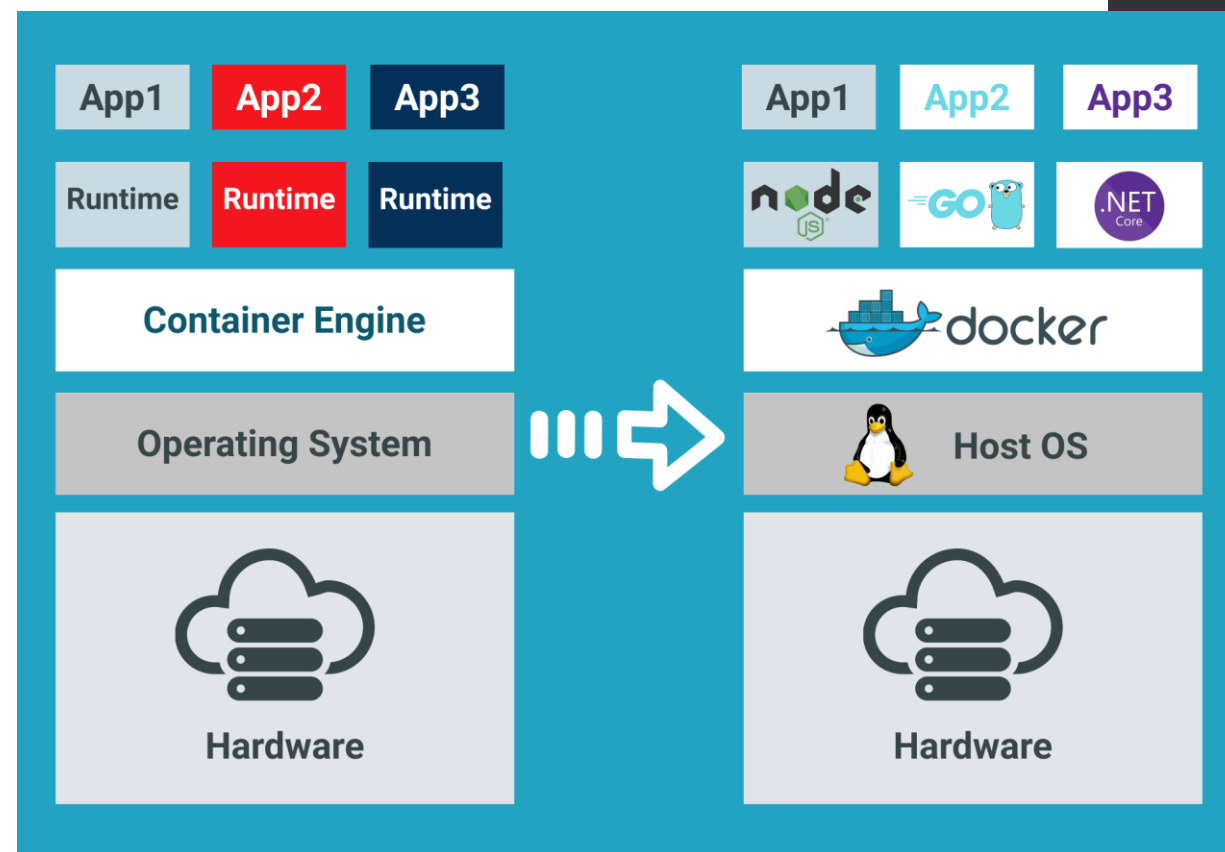
Puppet: Ayuda a automatizar la implementación y la gestión de la configuración del software.

Herramientas:

Contenedores:

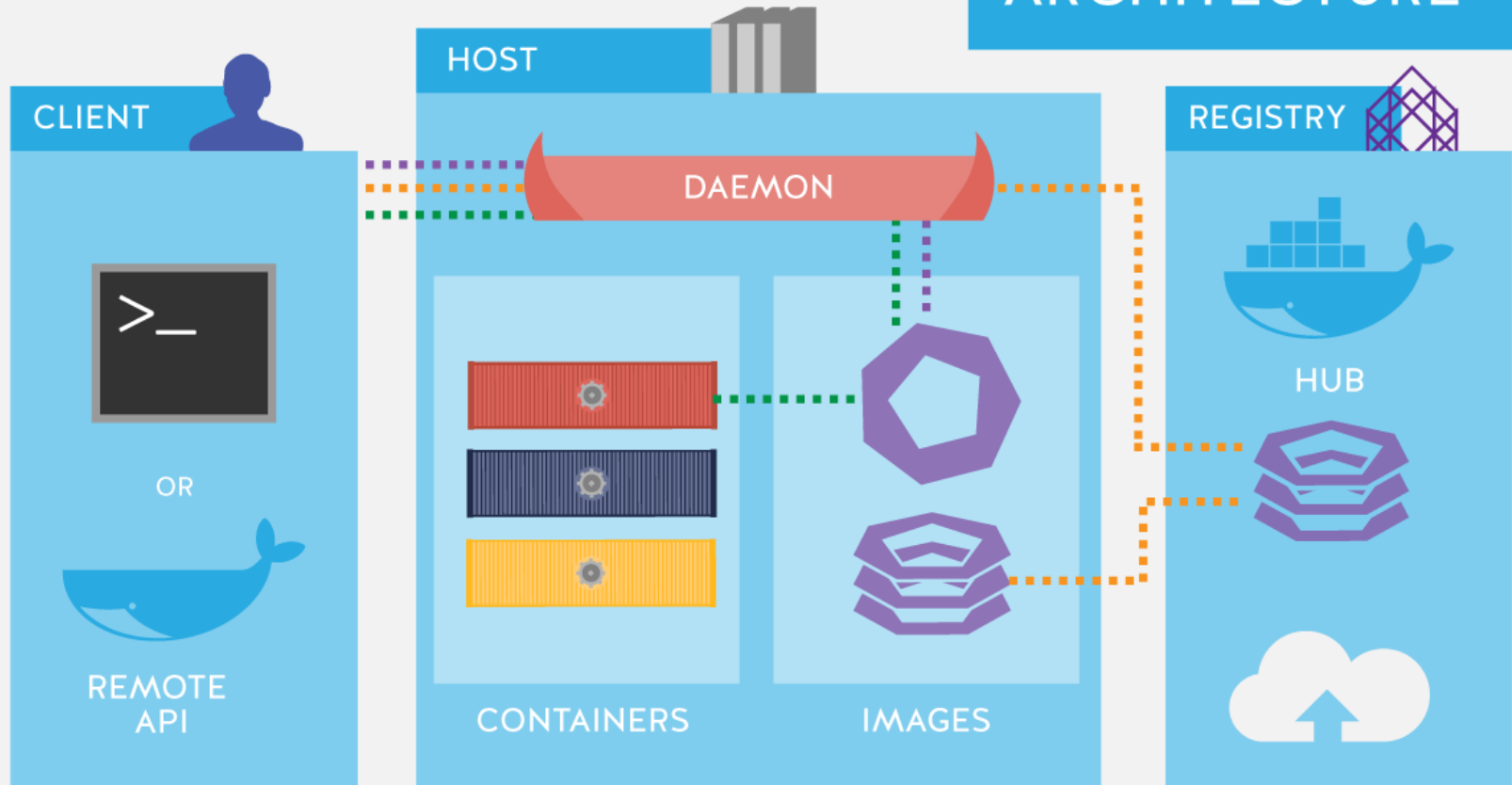
Los contenedores, especialmente con Docker, pueden utilizarse para encapsular aplicaciones y sus dependencias, lo que facilita su implementación y escalabilidad.

Puedes crear imágenes de contenedor preconfiguradas que incluyan el software necesario y luego implementarlas en cualquier lugar que ejecute Docker.

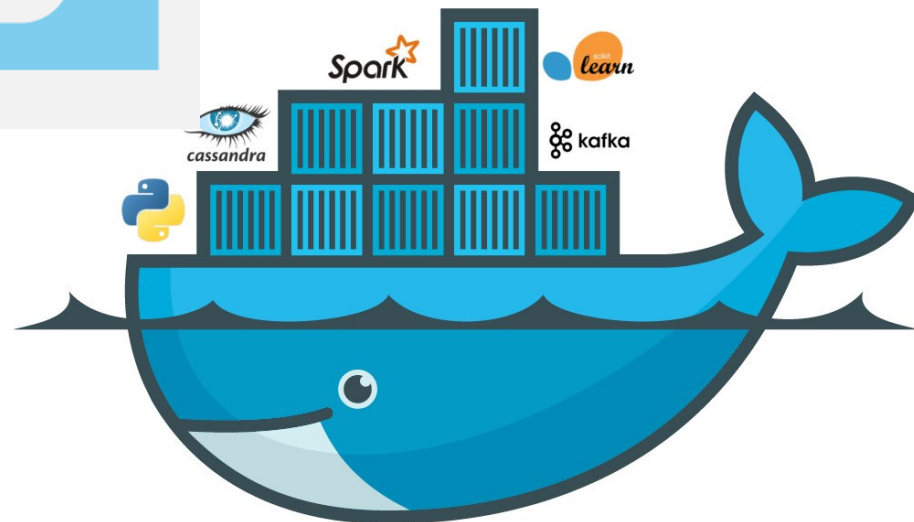


BUILD PULL RUN

DOCKER ARCHITECTURE



NORDICAPIS.COM



Herramientas:

Imágenes de sistema:

Las imágenes de sistema son instantáneas de un sistema operativo con un conjunto específico de software instalado y configurado. Puedes crear imágenes personalizadas para tu infraestructura y luego implementarlas en nuevos sistemas de manera rápida y consistente.



Gestores de paquetes y repositorios:

Utiliza gestores de paquetes y repositorios de software para simplificar y automatizar la instalación de software en sistemas operativos específicos.

Pruebas y Solución de Problemas



Pruebas y la solución de problemas

Las pruebas y la solución de problemas son pasos críticos en cualquier proceso de automatización para garantizar que los scripts y las configuraciones funcionen como se esperaba.

Pruebas unitarias:

Verificación de sintaxis: Antes de ejecutar un script o un playbook, verifica la sintaxis para evitar errores de sintaxis.

Pruebas en entornos de desarrollo: Realiza pruebas iniciales en entornos de desarrollo o máquinas virtuales para garantizar que los scripts funcionen según lo previsto antes de aplicarlos en entornos de producción.

Pruebas de integración:

Simulación de escenarios reales:

Realiza pruebas que simulen condiciones y configuraciones del mundo real para identificar posibles problemas.

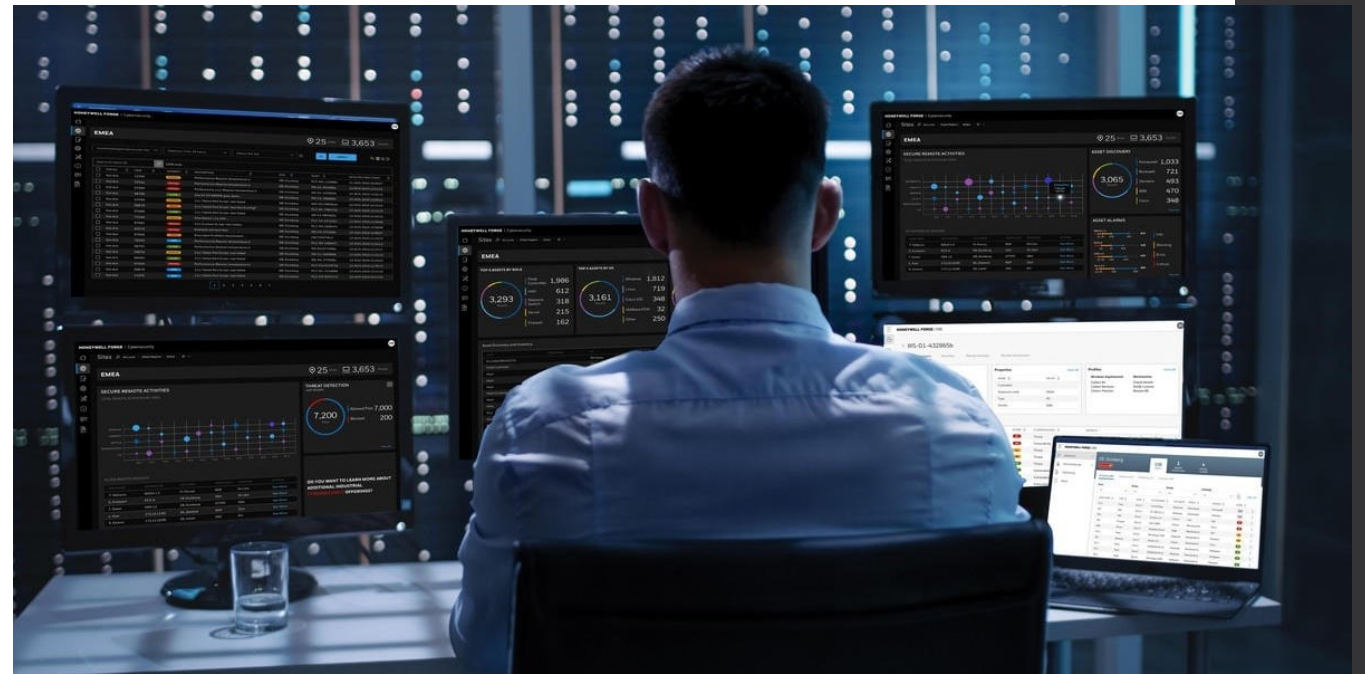
Pruebas de extremo a extremo: Ejecuta pruebas que abarquen todo el flujo de trabajo, desde el aprovisionamiento hasta la configuración y la aplicación de cambios.

Pruebas y la solución de problemas

Registro y monitoreo:

Registros detallados: Asegúrate de que los scripts y las herramientas de automatización generen registros detallados para facilitar la identificación y solución de problemas.

Monitoreo continuo: Implementa sistemas de monitoreo para detectar problemas de manera proactiva y solucionarlos antes de que afecten a los usuarios finales.



Pruebas y la solución de problemas

Solución de problemas:

Identificación de errores: Analiza los registros y los mensajes de error para identificar la causa raíz de los problemas.

Revisión de cambios recientes: Si algo dejó de funcionar después de una actualización o cambio, revisa los cambios recientes para determinar si están relacionados con el problema.

Pruebas de diagnóstico: Realiza pruebas de diagnóstico para aislar el problema y determinar qué parte del sistema está causando el fallo.



Pruebas y la solución de problemas

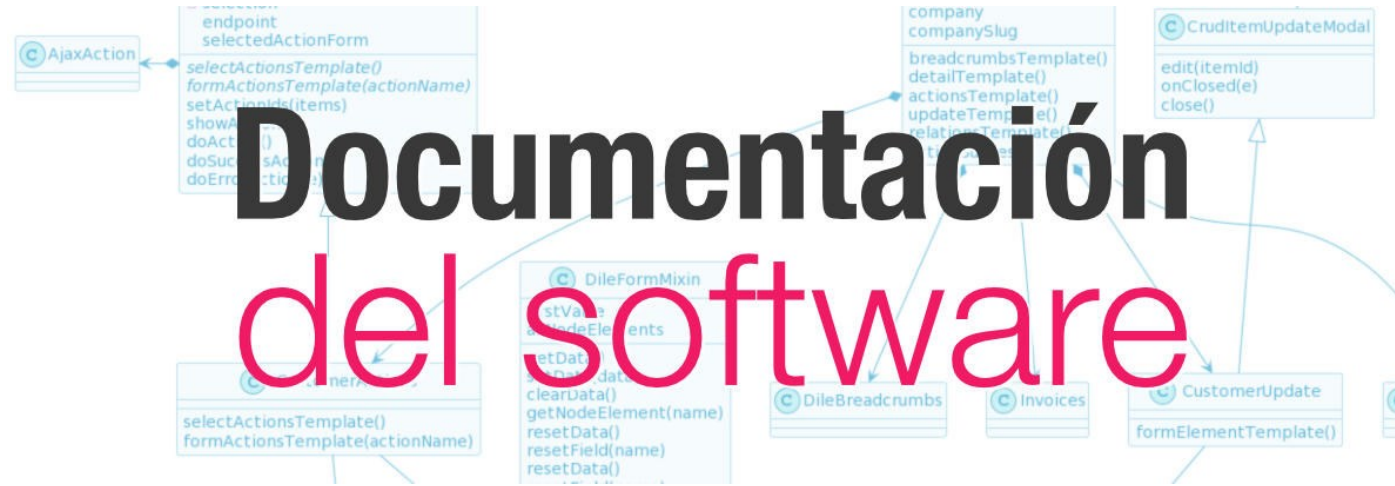
Mejora continua:

Retroalimentación y revisión: Solicita comentarios de los usuarios y realiza revisiones periódicas para mejorar y optimizar los scripts y las configuraciones de automatización.

Actualizaciones y mantenimiento: Mantén tus scripts y herramientas de automatización actualizados para evitar problemas de seguridad y compatibilidad.



Pruebas y la solución de problemas

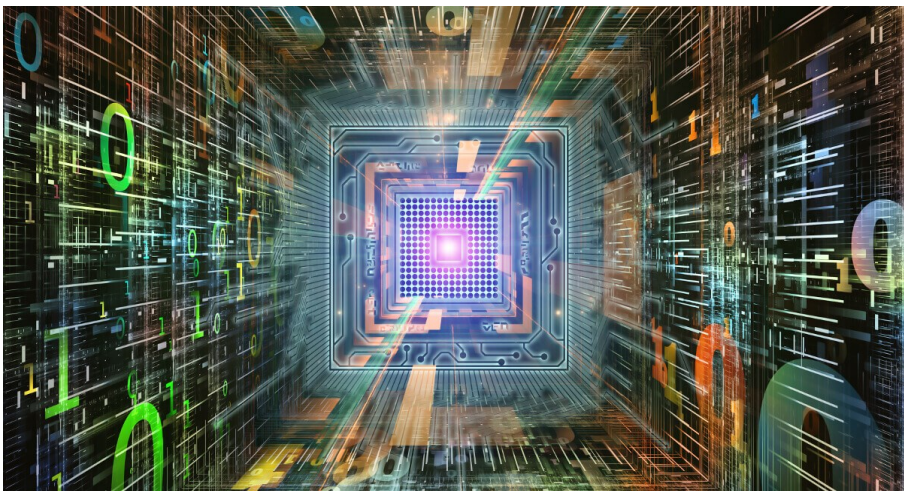


Documentación:

Documentación clara y concisa: Documenta los procedimientos de instalación, configuración y solución de problemas para facilitar la colaboración y el mantenimiento.

Notas de versión: Mantén actualizada la documentación con notas de versión que describan los cambios realizados en cada versión de los scripts y las configuraciones.

Performance, Reliability y Balanceo de Carga

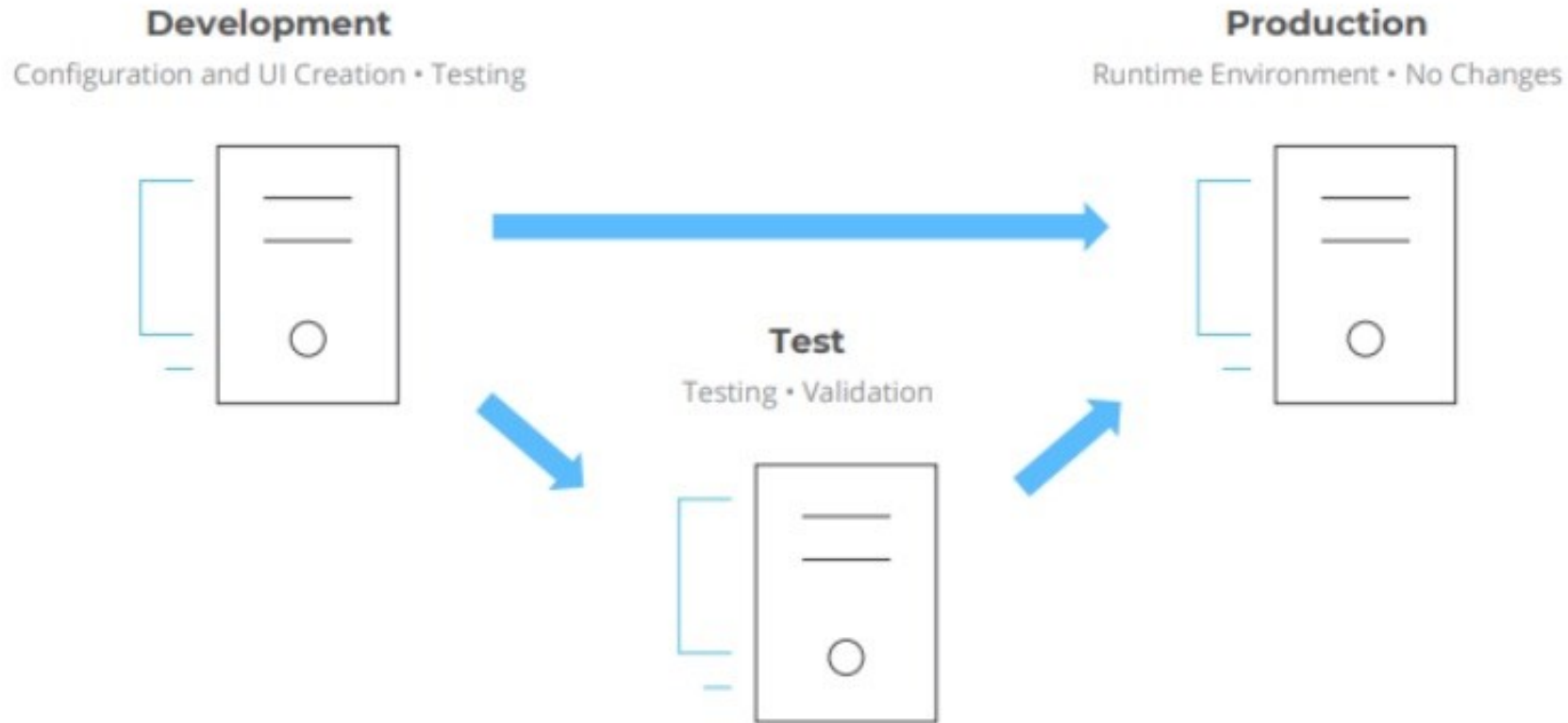


Aspecto	Performance	Fiabilidad	Balanceo de Carga
Definición	Medida de la velocidad y eficiencia del sistema.	Capacidad del sistema para operar sin fallos.	Distribución equitativa de carga entre servidores.
Objetivo	Mejorar la velocidad y eficiencia de los sistemas	Mantener la disponibilidad y consistencia de los sistemas	Distribuir equitativamente las solicitudes entre servidores.
Principales consideraciones	Optimización de recursos, ajuste de parámetros, caching, paralelización	Respaldo y recuperación, alta disponibilidad, pruebas de tolerancia a fallos	Distribución de carga, escalabilidad horizontal y vertical, monitoreo y ajuste
Herramientas y técnicas	Herramientas de monitoreo, ajuste de parámetros, estrategias de caché, paralelización	Plan de respaldo y recuperación, sistemas redundantes, pruebas de tolerancia a fallos	Algoritmos de balanceo de carga, escalabilidad horizontal y vertical, automatización de procesos
Beneficios	Mejora la velocidad de procesamiento y la eficiencia de los sistemas.	Minimiza el tiempo de inactividad y protege los datos críticos.	Distribuye equitativamente la carga para mejorar el rendimiento y la estabilidad.
Riesgos	Posible sobrecarga de recursos, subutilización de recursos.	Falta de planificación de respaldo y recuperación, tiempo de inactividad prolongado.	Sobrecarga de servidores, distribución desigual de la carga.

Plan de
operaciones.
Entornos de
desarrollo, testing
y producción.



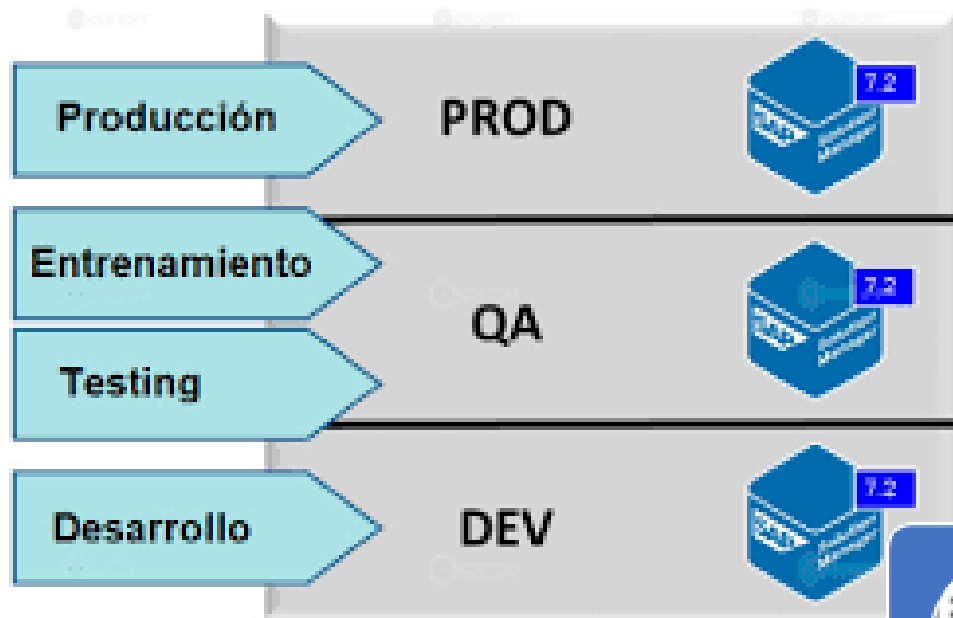
Plan de operaciones



Un plan de operaciones es un documento detallado que describe cómo se ejecutarán y gestionarán las actividades en diferentes entornos, como desarrollo, testing y producción, para garantizar una implementación exitosa y un funcionamiento continuo del sistema.

Desarrollo, Testing y Producción

Landscape con 3 ambientes

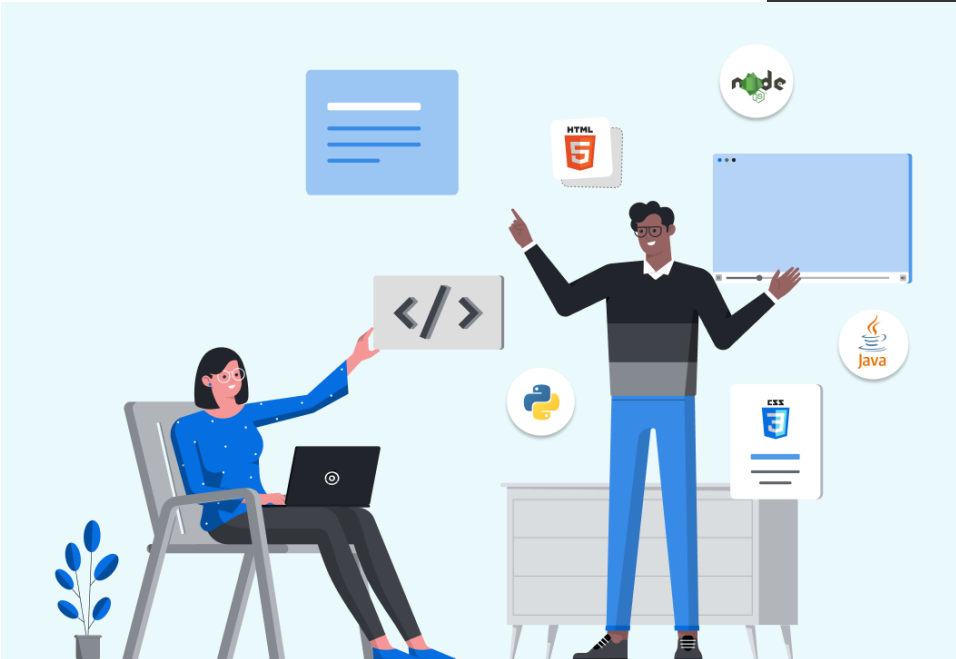


Ejemplos



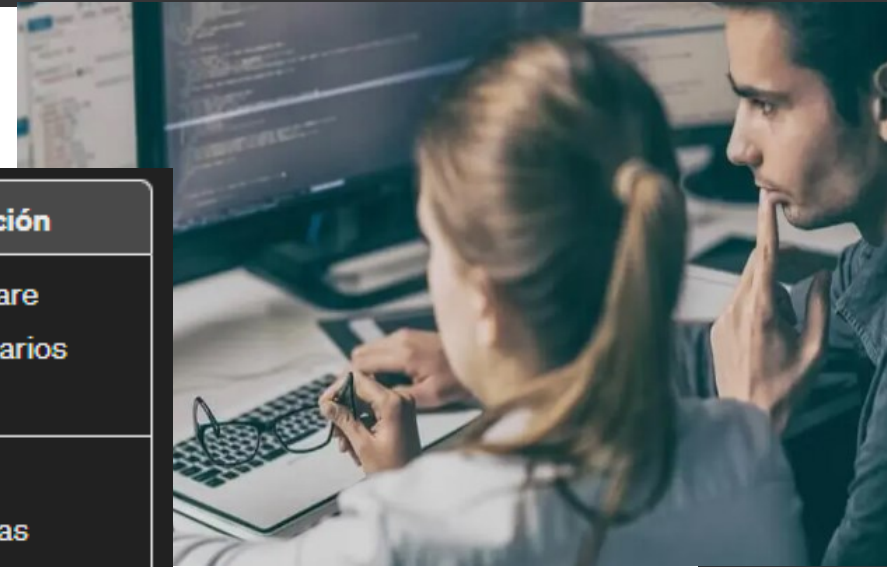
Aspecto	Entorno de Desarrollo	Entorno de Testing	Entorno de Producción
Propósito	Desarrollo inicial de aplicaciones y sistemas.	Realizar pruebas exhaustivas del software.	Ejecución del software disponible para usuarios finales.
Actividades	Desarrollo de código, integración continua, pruebas iniciales.	Pruebas de sistema, rendimiento, seguridad.	Implementación de nuevas versiones, monitoreo, resolución de incidentes.
Infraestructura	Ambientes individuales para desarrolladores, servidores de desarrollo.	Servidores de prueba replicando entorno de producción.	Servidores altamente disponibles y escalables, balanceadores de carga.
Pruebas	Pruebas unitarias, pruebas de integración inicial.	Pruebas de sistema, aceptación del usuario, rendimiento.	Pruebas de rendimiento, seguridad, escalabilidad.
Herramientas	Git, IDEs, herramientas de integración continua.	Herramientas de automatización de pruebas, monitoreo.	Herramientas de monitoreo, balanceadores de carga.
Gestión de Configuración	Utilización de herramientas de gestión de versiones.	Implementación de herramientas de gestión de configuración.	Automatización de despliegues, gestión de configuraciones.
Seguridad	Consideraciones básicas de seguridad para desarrollo seguro.	Pruebas de seguridad, análisis de vulnerabilidades.	Implementación de medidas de seguridad robustas.
Documentación	Documentación de código, procedimientos de desarrollo. 	Documentación de pruebas realizadas, resultados.	Documentación de procedimientos, configuraciones.

Desarrollo, Testing y Producción



Resumen:

Aspecto	Entorno de Desarrollo	Entorno de Testing	Entorno de Producción
Propósito	Desarrollo inicial de aplicaciones y sistemas.	Realizar pruebas exhaustivas del software.	Ejecución del software disponible para usuarios finales.
Actividades	Desarrollo de código y nuevas características. Integración continua y entrega continua (CI/CD). Pruebas unitarias y pruebas de integración iniciales.	Pruebas de sistema y pruebas de aceptación del usuario (UAT). Pruebas de rendimiento, carga y estrés. Pruebas de seguridad y pruebas de penetración.	Implementación y despliegue de nuevas versiones del software. Monitoreo del rendimiento, disponibilidad y seguridad del sistema. Resolución de incidentes y problemas de producción.
Infraestructura	Ambientes individuales para desarrolladores. Servidores de desarrollo para integración y pruebas. Herramientas de gestión de versiones como Git.	Servidores de prueba replicando el entorno de producción. Herramientas de automatización de pruebas como Selenium, JMeter.	Servidores de producción altamente disponibles y escalables. Balanceadores de carga y servicios de CDN. Herramientas de monitoreo y registro como Nagios, Prometheus, ELK Stack.



Disponibilidad de
software, continuidad y
niveles de servicio.
Dimensionamiento y
gestión de capacidad



Disponibilidad de software, continuidad y niveles de servicio.

Disponibilidad de Software:

Capacidad del sistema para estar disponible y accesible para los usuarios en todo momento. Esto implica que el software esté funcionando correctamente y que los usuarios puedan acceder a él cuando lo necesiten.

Aspectos clave de la disponibilidad de software incluyen:

- *Tiempo de actividad*: Se refiere al tiempo durante el cual el software está disponible y operativo.
- *Tiempo de inactividad*: Se refiere al tiempo durante el cual el software no está disponible debido a fallos o mantenimiento planificado.
- *Resiliencia*: La capacidad del software para recuperarse rápidamente de fallos y mantener la disponibilidad incluso en condiciones adversas.

Disponibilidad de software, continuidad y niveles de servicio.

Continuidad del Servicio:

Seguir funcionando de manera ininterrumpida incluso en situaciones de emergencia o crisis. Esto implica tener planes y procedimientos en su lugar para garantizar la continuidad de las operaciones y minimizar el impacto en caso de eventos inesperados, como desastres naturales, ciberataques o fallos de hardware.

Algunos aspectos clave de la continuidad del servicio incluyen:

- **Plan de Continuidad del Negocio (BCP):** Un plan detallado que describe cómo la organización responderá y se recuperará de eventos disruptivos.
- **Recuperación ante desastres (DR):** Procedimientos y recursos para restaurar sistemas críticos y datos en caso de un desastre.
- **Pruebas y simulacros:** Realización periódica de pruebas y simulacros para garantizar que los planes de continuidad del servicio sean efectivos y estén actualizados.



Niveles de Servicio (SLA):

Los niveles de servicio (SLA) son acuerdos formales entre proveedores de servicios y clientes que definen los estándares de rendimiento esperados y los compromisos de servicio.

Estos acuerdos especifican métricas como el tiempo de actividad, el tiempo de respuesta y el tiempo de resolución de problemas, y establecen las consecuencias por incumplimiento.

Dimensionamiento y la gestión de capacidad

El dimensionamiento y la gestión de capacidad son procesos críticos para garantizar que los sistemas y recursos informáticos satisfagan las necesidades actuales y futuras de una organización de manera eficiente y efectiva.





Dimensionamiento

Implica determinar los recursos necesarios para que un sistema funcione correctamente en función de la carga de trabajo y los requisitos de rendimiento.

- *Análisis de la carga:* Evaluar la demanda actual y proyectada del sistema, incluyendo el tráfico de usuarios, la cantidad de datos procesados y cualquier otro factor que afecte la carga del sistema.
- *Estimación de recursos:* Determinar la cantidad de recursos informáticos necesarios, como CPU, memoria, almacenamiento y ancho de banda de red, para satisfacer la carga de trabajo prevista.
- *Planificación de la capacidad:* Identificar y provisionar los recursos necesarios para satisfacer los requisitos de rendimiento, garantizando al mismo tiempo un margen de seguridad para manejar picos de demanda inesperados.

Gestión de Capacidad

Implica monitorear, controlar y optimizar el uso de recursos informáticos para garantizar que se utilicen de manera eficiente y se satisfagan las necesidades operativas de la organización.

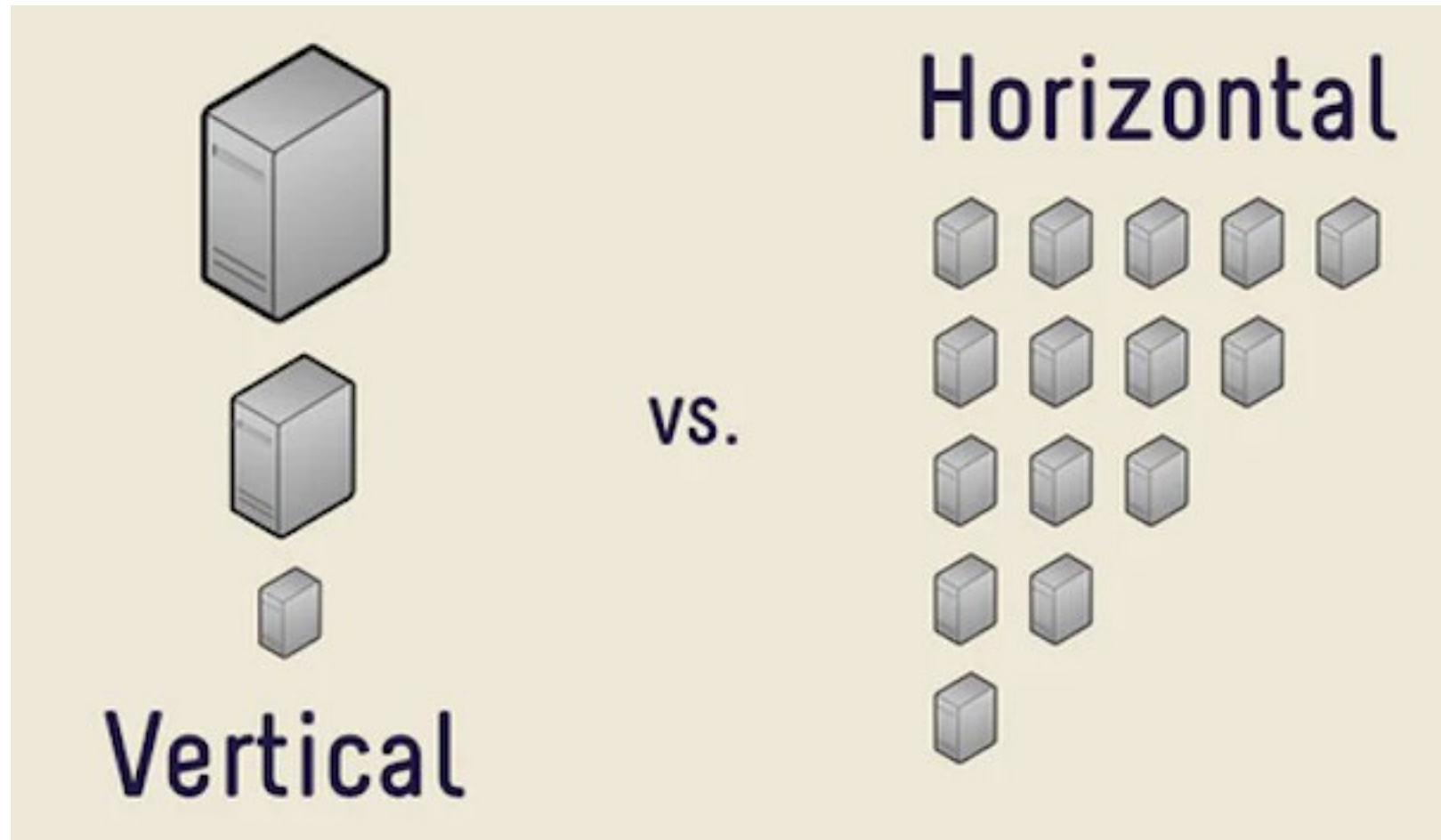


- *Monitoreo del rendimiento*
- *Planificación de la capacidad*
- *Optimización de recursos*
- *Escalabilidad: Implementar estrategias de escalabilidad, tanto horizontal como vertical, para asegurar que los sistemas puedan crecer y adaptarse a medida que cambian las demandas del negocio.*

Anexo: Escalabilidad Horizontal vs Vertical

Aspecto	Escalabilidad Horizontal	Escalabilidad Vertical
Definición	Agregar más instancias idénticas del sistema.	Aumentar la capacidad de un único servidor o instancia.
Aumento de recursos	Aumenta la capacidad agregando más servidores o nodos.	Aumenta la capacidad agregando más recursos a una instancia.
Distribución de carga	La carga de trabajo se distribuye entre las instancias.	El sistema opera con mayor capacidad en una sola instancia.
Flexibilidad	Mayor flexibilidad, se pueden agregar más instancias según sea necesario.	Menos flexibilidad, la escalabilidad está limitada a los recursos de una sola instancia.
Gestión y complejidad	Puede requerir más infraestructura y gestión debido al número de instancias.	Puede ser más simple de gestionar, ya que implica ajustar recursos en una sola instancia.
Costo	Puede tener un costo más bajo en términos de hardware individual.	Puede ser más costoso en términos de hardware, especialmente para actualizar hardware existente.
Ejemplo	Agregar más servidores web y distribuir la carga entre ellos.	Aumentar la cantidad de RAM en un servidor existente.

Anexo: Escalabilidad Horizontal vs Vertical

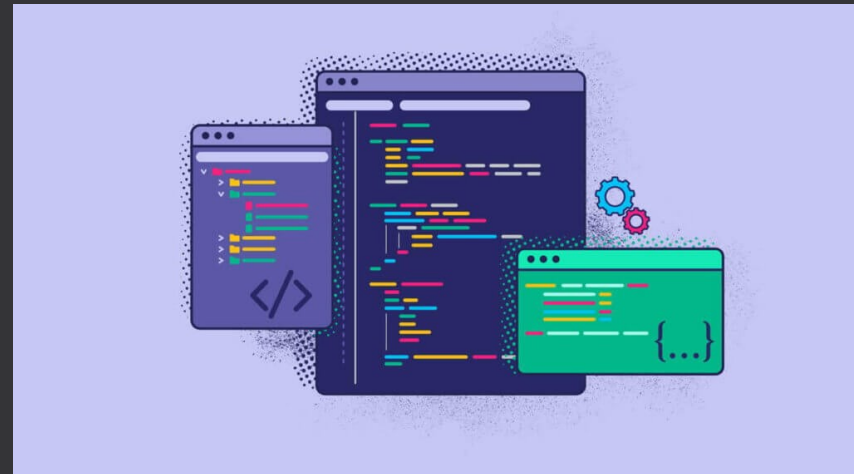


Ejercicios en Grupo

EJERCICIO EN EQUIPO Nro. 03:

Realizar un scripting con algunas de las herramientas mencionadas en clase, y propongan un pequeño script de automatización para un caso de aplicación propuesto por el grupo.

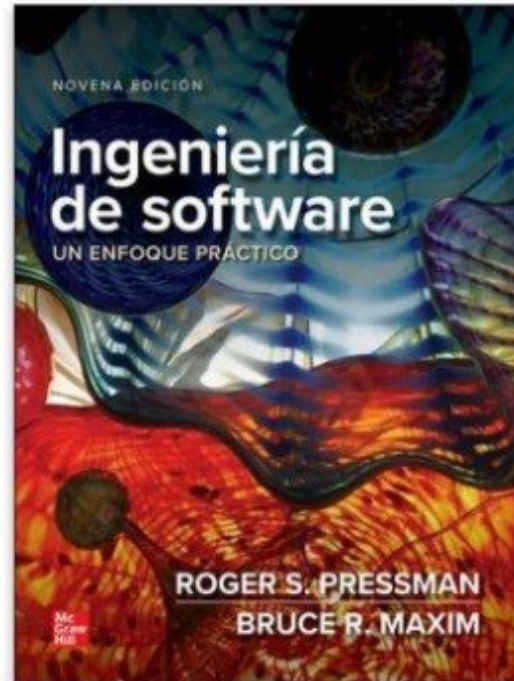
Esto será comentado la próxima clase, en no más de 5 min por equipo. Podrán ayudarse con un pdf/ppt y/o muestra del script.



Bibliografía

Bibliografía (Complementaria)

Pressman, Roger S. (2021). Ingeniería de Software: Un Enfoque Práctico (9a ed.). Ciudad de México: McGraw-Hill.



INGENIERIA DE SOFTWARE

9th Edición
1456286889 · 9781456286880

Autor(es) Roger S. Pressman
© 2021 | Published: April 18, 2021

Este libro presenta un marco de trabajo que puede ser utilizado por todos los desarrolladores de software de computadora. El marco de trabajo abarca un proceso, un conjunto de métodos y una colección de herramientas que llamamos ingeniería de soft... [Leer Más](#)

♡ Añadir a la Lista de Deseos

Bibliografía (Complementaria)

*Guide to the Software
Engineering Body of Knowledge*

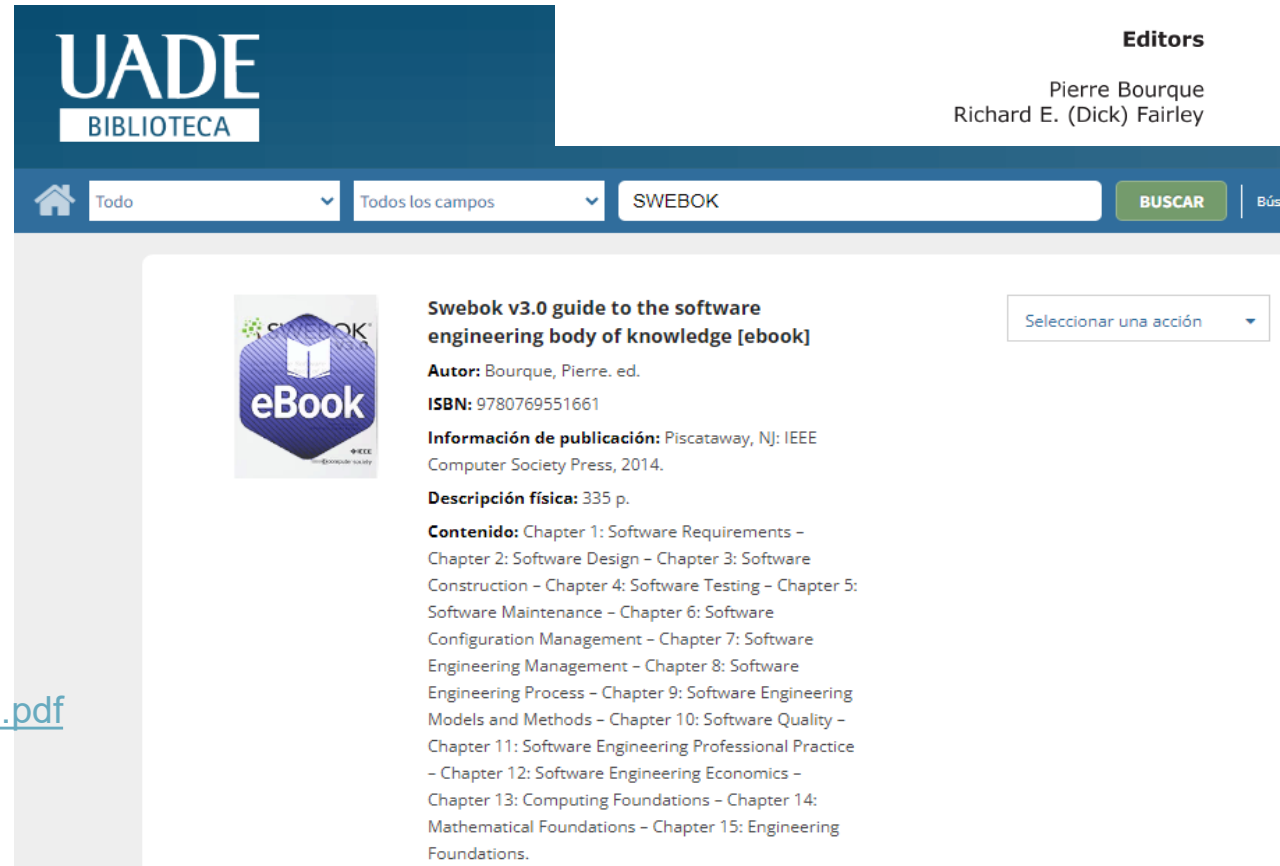
Swebok v3.0 guide to the software
engineering body of knowledge [ebook]

Autor: Bourque, Pierre. ed. ISBN:
9780769551661

Información de publicación: Piscataway,
NJ: IEEE Computer Society Press, 2014.

Capítulo 5 y 7

<https://ieeecs-media.computer.org/media/education/swebok/swebok-v3.pdf>



UADE
BIBLIOTECA

Editors
Pierre Bourque
Richard E. (Dick) Fairley

Todo Todos los campos SWEBOK **BUSCAR**

 **Swebok v3.0 guide to the software engineering body of knowledge [ebook]** Seleccionar una acción

Autor: Bourque, Pierre. ed.
ISBN: 9780769551661
Información de publicación: Piscataway, NJ: IEEE Computer Society Press, 2014.
Descripción física: 335 p.
Contenido: Chapter 1: Software Requirements – Chapter 2: Software Design – Chapter 3: Software Construction – Chapter 4: Software Testing – Chapter 5: Software Maintenance – Chapter 6: Software Configuration Management – Chapter 7: Software Engineering Management – Chapter 8: Software Engineering Process – Chapter 9: Software Engineering Models and Methods – Chapter 10: Software Quality – Chapter 11: Software Engineering Professional Practice – Chapter 12: Software Engineering Economics – Chapter 13: Computing Foundations – Chapter 14: Mathematical Foundations – Chapter 15: Engineering Foundations.

SWEBOK GUIDE - ESTRUCTURA

Capítulo 1:
Requisitos de
software

Capítulo 2: Diseño
de software

Capítulo 3:
Construcción de
software

Capítulo 4:
Pruebas de
software

**Capítulo 5:
Mantenimiento de
software**

Capítulo 6: Gestión
de la configuración
de software

**Capítulo 7: Gestión
de ingeniería de
software**

Capítulo 8:
Proceso de
ingeniería de
software

Capítulo 9:
Modelos y
métodos de
ingeniería de
software

Capítulo 10:
Calidad del
software

Capítulo 11:
Práctica
profesional de la
ingeniería de
software

Capítulo 12:
Economía de la
ingeniería de
software

Capítulo 13:
Fundamentos de la
informática

Capítulo 14:
Fundamentos
matemáticos

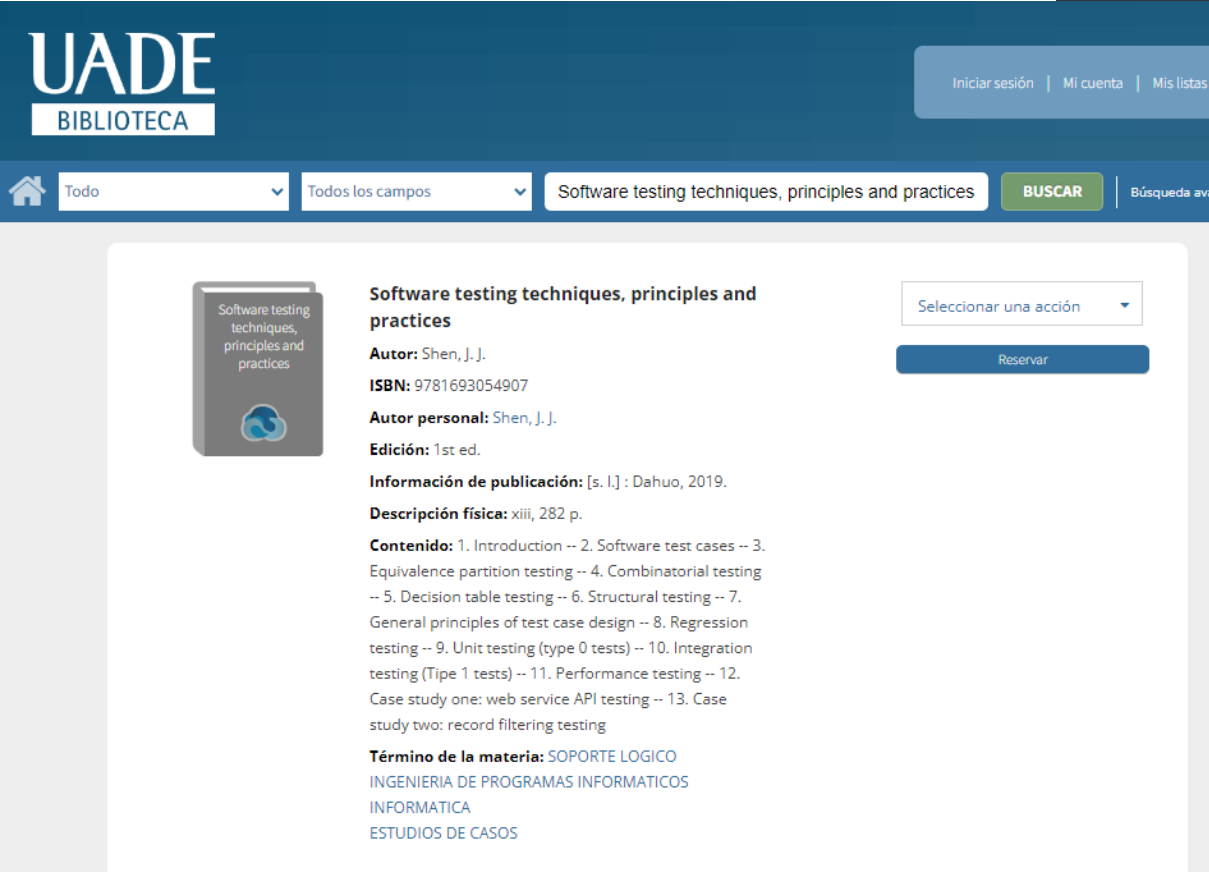
Capítulo 15:
Fundamentos de la
ingeniería.

Bibliografía (Complementaria)

Shen, J. J. (2019). Software Testing Techniques, Principles and Practices (1st ed.). [s. l.]: Dahuo.

Link de acceso Biblioteca UADE:

[https://biblioteca.uade.edu.ar/client/es_ES/biblioteca/search/detailnonmodal/ent:\\$002f\\$002fSD_ILS\\$002f0\\$002fSD_ILS:322387/one?qu=Software+testing+techniques%2C+principles+and+practices](https://biblioteca.uade.edu.ar/client/es_ES/biblioteca/search/detailnonmodal/ent:$002f$002fSD_ILS$002f0$002fSD_ILS:322387/one?qu=Software+testing+techniques%2C+principles+and+practices)



The screenshot displays the UADE Biblioteca website interface. At the top, the UADE logo and 'BIBLIOTECA' are visible. Navigation links for 'Iniciar sesión', 'Mi cuenta', and 'Mis listas' are in the top right. A search bar contains the text 'Software testing techniques, principles and practices', with a 'BUSCAR' button and a 'Búsqueda avanzada' link. Below the search bar, the book details are shown. On the left is a book cover thumbnail. To the right, the title 'Software testing techniques, principles and practices' is displayed, followed by the author 'Shen, J. J.', ISBN '9781693054907', and edition '1st ed.'. The publication information states '[s. l.] : Dahuo, 2019.' and the physical description is 'xiii, 282 p.'. The content summary lists 13 topics: 1. Introduction, 2. Software test cases, 3. Equivalence partition testing, 4. Combinatorial testing, 5. Decision table testing, 6. Structural testing, 7. General principles of test case design, 8. Regression testing, 9. Unit testing (type 0 tests), 10. Integration testing (Type 1 tests), 11. Performance testing, 12. Case study one: web service API testing, and 13. Case study two: record filtering testing. The subject terms are listed as 'SOPORTE LOGICO', 'INGENIERIA DE PROGRAMAS INFORMATICOS', 'INFORMATICA', and 'ESTUDIOS DE CASOS'. Action buttons for 'Seleccionar una acción' and 'Reservar' are on the right.



Dra. Ing. Roxana Martínez